AI Video Dubbing — Project Write-up

AL-AI-06 | Translate and Re-Voice Short Videos

Candidate: Sk Khaja Mainuddin Role: AI Application Developer Date: 24 October 2026

1. Summary

I built a Streamlit application that takes a short video, prepares its audio for speech processing, translates recognized dialogue into a selected language, generates replacement speech, and combines the dub with the video. The workflow uses AudicLabs audio services, a configurable translation adapter, text-to-speech, Demucs source separation, and FFmpeg for media assembly. The single-video path and local recovery of the sample output were verified. Batch processing is implemented in the interface and code; a complete batch run has not been confirmed.

2. Architecture

Input Video upload and batch selection	Audio analysis FFmpeg extraction, denoising, diarization and transcription
Language layer Timestamped segments, translation adapter and fallback models	Speech layer TTS, speaker voice assignment and segment-duration fitting
Assembly Demucs vocal/background separation, dub mix and FFmpeg mux	Outputs Dubbed MP4, translated SRT, JSON timing report and UI preview

3. Key design decisions

• Replaceable translation: a dedicated adapter keeps provider configuration and fallback models separate from the main pipeline.

• Timestamped segments: generated speech is fitted to each segment slot, and timing values are saved for evaluation.

• Background preservation: Demucs estimates vocals and accompaniment so music and effects can be retained while dialogue is replaced.

• Local media processing: FFmpeg handles extraction and muxing; Pydub manages audio timeline placement and mixing.

4. Results

The saved report for the 125.614-second sample records 38 speech segments and 23 speed adjustments. Across all 38 segments with timing measurements, mean absolute duration difference was 0.103 seconds and maximum absolute difference was 1.185 seconds. The automated test suite completed with 22 passed. The recovered MP4 was validated with FFprobe as H.264 video plus AAC audio, duration 125.56 seconds, and played back successfully.

Reproduction: activate the project virtual environment, install pinned dependencies, run **python -m pytest -v**, start the Streamlit app, and process a sample with valid API configuration. Timing figures come from **samples/output/0cc069aba2b5_report.json** and the evaluation module. No independent latency or cost benchmark is claimed.

5. Requirement coverage

Requirement area	Status	Evidence / notes
A. Analyse the Source	**Verified in code**	Extraction, denoising, transcription and diarization are wired into the pipeline. Broader tests with varied input videos remain useful.
B. Translate for Timing	**Implemented**	Translation adapter and fallback handling are present; timestamped segments and timing metrics are saved. Provider and language coverage needs broader testing.
C. Re-Voice and Assemble	**Verified on sample**	TTS, voice assignment, fitting, Demucs separation and FFmpeg muxing are implemented. The recovered sample was validated and played back; quality may vary by source.
D. Interface and Evaluation	**Implemented; partly tested**	Streamlit includes single-video, batch and API diagnostics views, previews and downloads. Batch end-to-end validation and structured independent ratings remain outstanding.
Additional requirements	**Core items present**	Pinned dependencies, example environment, README, automated tests and sample artifacts are present. The demo recording and final submission checklist should be confirmed before submission.

6. Known limitations

Demucs separation is an estimate: original speech can leak into the background stem, and music/effects can leak into the vocal stem. Speed adjustment can affect naturalness, while timing metrics do not measure translation accuracy or perceived synchronization. Supported languages and voices depend on configured services. The sample was checked locally, but batch processing has not been validated end to end. Listening feedback for three outputs was informal rather than a blinded or independently scored evaluation.

7. What I would do next

I would first run a clean-clone setup and a controlled batch test with valid and invalid videos, recording per-file outcomes and a batch summary. Next, I would add repeatable evaluation for translation quality, speech intelligibility, synchronization and background preservation using a consistent rating rubric. I would also strengthen temporary-file cleanup and output validation so a failed mux cannot leave a corrupt MP4 marked as complete.

Prepared for the AudicLabs India AL-AI-06 assessment. Status labels distinguish implemented components from results verified on the sample.